FinPulse

How it works, how to run it, and how to explain it

This guide is written in simple English. Every technical word is explained the first time it is used. Nothing has been left out to make it easier — the hard parts are still here, just explained slowly.

Part	What it covers
1	What the app does, in one page
2	The one big idea behind the design
3	The exact order things happen in
4	The tools and libraries used
5	Getting the company data
6	All 11 ratios, explained from zero
7	How we decide strong, adequate or weak
8	Trends, and two real bugs
9	The handwritten notes and how we search them
10	Talking to the AI
11	The number guard
12	Saving answers so we do not repeat work
13	The screen
14	Testing: what a test harness is, and every command
15	Running it on your own computer
16	Putting it online
17	Is this harness engineering?
18	30 interview questions with answers
19	Glossary

Live at fin-pulse-financial-health-explaine.vercel.app

Part 1 — What the app does

The problem

Every company that sells shares to the public has to publish its accounts. Those documents are long and full of accounting words. A normal person cannot read one and answer a simple question: **is this company doing well or badly?**

Other websites show you financial ratios, but they only show the number. Seeing **current ratio: 0.89** tells you nothing unless you already know what a current ratio is, what a good one looks like, and whether 0.89 is bad for that kind of business.

What FinPulse does instead

You type a company's short stock-market code — its **ticker**, like AAPL for Apple. The app then:

1. Downloads four years of that company's real accounts.
2. Works out eleven standard financial ratios.
3. Decides whether each one is strong, adequate or weak.
4. Asks an AI to write two or three sentences explaining each number.
5. Checks that the AI did not make any numbers up.

What you see

```
NVIDIA Corporation  [NVDA]
Technology - Semiconductors - Market cap $4385.1B

 Liquidity        Profitability      Leverage        Cash
 Adequate         Strong             Strong          Strong

 Liquidity  [Adequate]

 Current ratio                    3.91   /\__   down  Improving
 NVIDIA's current ratio of 3.91 indicates it has almost four dollars of
 current assets for each dollar of current liabilities. In the technology
 sector, such a high ratio is common because the company holds little
 inventory and large cash balances, so the level is not a red flag.
```

Four summary boxes at the top. Then each ratio with its value, a tiny four-year chart called a **sparkline**, an arrow showing the trend, and the explanation underneath.

The app never tells you to buy or sell anything. It says so on every page, and the AI is specifically told not to. It explains what the numbers mean. What you do with that is up to you.

Part 2 — The one big idea

> **The computer does the maths. The AI only does the words. And a checker sits between them.**

Almost every design choice in this project comes from that one sentence. It is also the best thing to say if someone asks you about the project.

Why it matters

AI language models are very good at writing smooth sentences. They are **not** reliable at arithmetic, and they have a well-known habit of stating numbers that are simply invented. The technical word for this is **hallucination**.

In a money app, an invented number is not a small problem. If the app says a company's profit margin is 40% when it is really 4%, that is worse than saying nothing, because you would believe it.

Most AI finance tools hand the model some raw data and say “analyse this.” The model then does two jobs at once — working out numbers and writing about them — and nobody can tell which figures are real.

So we split the work

The job	Who does it	Why
Download the accounts	Our code	Needs to be exact and repeatable
Work out the ratios	Our code	Arithmetic must never be guessed
Judge strong or weak	Our code	Fixed rules anyone can check
Write the explanation	The AI	This is language, its real strength
Check the explanation	Our code	Catch any number the AI invented

The AI is given **finished numbers** and told: explain these, invent nothing. Afterwards a piece of our code, the **number guard**, reads what the AI wrote, finds every number in it, and compares each one against the list of numbers the AI was actually given. If anything does not match, that whole paragraph is thrown away.

> You cannot make a model honest by asking politely. You can only check its work afterwards. That is why the checker runs **after** the AI writes, not before.

Part 3 — The exact order things happen in

This is the part people most often get wrong, so here it is slowly, with real numbers.

Two separate sources of information

Information reaches the AI from **two completely different places**:

	Source 1	Source 2
What it is	This company's actual numbers	General knowledge about ratios
Where it comes from	Yahoo Finance, downloaded live	42 text files you wrote by hand
Does it change per company?	Yes, every single time	No — the same 42 notes always
Example	"Current ratio: 0.87"	"Supermarkets can run below 1.0 safely"

The order, step by step

Say you type **WMT** (Walmart).

Step 1 — Download the raw numbers

Yahoo Finance sends back raw line items from the accounts. No ratios yet, just figures:

```
fiscalYear: 2026
currentAssets:       76_000_000_000
currentLiabilities:  92_000_000_000
inventory:           56_000_000_000
revenue:            681_000_000_000
...
```

The handwritten notes are **not involved at all** at this stage.

Step 2 — Do the maths

Our code divides the numbers to get ratios. Still no notes, still no AI:

```
current ratio = 76,000,000,000 / 92,000,000,000 = 0.83
quick ratio   = (76,000,000,000 - 56,000,000,000) / 92,000,000,000 = 0.22
```

It also compares four years to work out a trend, and applies the threshold rules to get a verdict:

```
current ratio: 0.83  ->  weak  ->  trend: stable
```

Step 3 — Use the result to search the notes

This is the step most people miss. The computed result is what we search with. The query is built from the ratio name, its value, its trend, the sector and the category:

```
"current ratio 0.83 stable trend consumer defensive sector liquidity"
```

That sentence is compared against all 42 handwritten notes, and the three closest ones are picked. Because the sector is in the query, the **supermarket exception** note wins here — which it would not if we had only searched for “current ratio”.

So the maths **must** happen before the search. The search needs the answer.

Step 4 — Now the two sources are merged

Only at this point do the company numbers and the handwritten notes meet, inside one piece of text called the **prompt**:

```
COMPANY: Walmart Inc. (Consumer Defensive)

COMPUTED RATIOS:                        <-- from step 2
- Current ratio: 0.83 (2026), trend: stable
- Quick ratio: 0.22 (2026), trend: stable

REFERENCE NOTES:                        <-- from step 3
[current-ratio] The current ratio measures whether a company can cover its
short-term obligations with short-term assets...
[current-ratio-sector-exceptions] Grocery and restaurant chains collect cash
from customers immediately but pay suppliers on 30-60 day terms, so they can
run current ratios below 1.0 indefinitely without distress...

Rules:
- Use only the numbers given above. Do not calculate anything.
- If the reference notes mention a sector exception that applies here, say so.
- Do not give investment advice or suggest buying or selling.
```

Step 5 — The AI writes

The AI reads that prompt and writes. It has the number (0.83), it has the reason that number is normal for a supermarket, and it has been told not to calculate anything. So it produces something like:

> “Walmart's current ratio of 0.83 means short-term bills slightly exceed short-term assets. For a grocery retailer this is normal rather than worrying, because customers pay immediately while suppliers are paid on 30 to 60 day terms. The stable trend suggests this is a settled way of operating, not a developing problem.”

Step 6 — The checker reads it back

Our code pulls out every number in that paragraph — 0.83, 30, 60 — and checks each one. 0.83 came from step 2. 30 and 60 came from the note in step 3. All accounted for, so the paragraph is kept.

If the AI had written “the industry average is 1.4”, that 1.4 would match nothing, and the whole paragraph would be deleted.

The corrected summary

```
Yahoo numbers
      |
      v
COMPUTE the ratios              <-- notes not involved yet
      |
      v
USE the result to search the handwritten notes
      |
      v
MERGE numbers + notes into one prompt
      |
      v
AI writes, using only what is in the prompt
      |
      v
CHECKER verifies every digit
```

> The common mistake is thinking the notes are merged in **before** the maths. They are merged in **after**, and the result of the maths is what chooses which notes to merge.

And yes — it writes strictly from the results

That is the whole point. The AI is given finished numbers and is told not to calculate. If it ignores that and produces a number of its own, the checker catches it and that explanation is dropped. The reader then sees the ratio with no text underneath, which is a worse page but never a wrong one.

Part 4 — The tools we used

Tool	What it is	Why we used it
TypeScript	JavaScript that checks types	Catches mistakes before the code runs
Next.js 16	A framework for building websites	Handles pages, server rendering, caching
React 19	Builds what you see on screen	Lets the page load in two stages
CSS Modules	Styling, one file per component	No extra library; styles cannot clash
yahoo-finance2	Downloads company accounts	Free, no sign-up, covers US companies
OpenAI SDK	Talks to the AI service	Many AI providers accept this same format
Groq	Runs the AI model	Free tier, very fast
Jina AI	Turns text into numbers	Needed to search the handwritten notes
Vitest	Runs our tests	Fast, understands TypeScript directly
Vercel	Hosts the website	Rebuilds automatically when we push code

How the folders are arranged

```
FinPulse/
  app/                 the website
    page.tsx             home page with the search box
    [ticker]/page.tsx    the report page for one company
    layout.tsx           wraps every page, holds the dark-mode script
    components/          the visual pieces
    actions.ts           handles the search form
  lib/                 all the thinking, no website code
    yahoo.ts             downloads the accounts
    ratios.ts            works out the 11 ratios
    health.ts            decides strong / adequate / weak
    retrieve.ts          picks the 3 best handwritten notes
    embed.ts             turns text into numbers
    llm.ts               talks to the AI
    explain.ts           builds the prompt and runs the checker
    format.ts            makes numbers readable
    cachedData.ts        saves answers so we do not repeat work
  content/kb/          42 handwritten notes
  content/kb-index.json  those notes turned into numbers
  __tests__/           77 tests in 5 files
  scripts/             build-kb-index.ts
```

Why lib/ is kept separate

Everything inside `lib/` is plain TypeScript that knows nothing about the website framework. Only one file, `lib/cachedData.ts`, mentions Next.js.

The benefit is that the thinking parts can be used from anywhere — a script, a test, or a different program entirely — without dragging a whole web framework along. It also makes the tests simple, because they can

import `lib/ratios.ts` directly and call it.

Part 5 — Getting the company data

File: `lib/yahoo.ts`

```
const [timeSeries, summary] = await Promise.all([
  yf.fundamentalsTimeSeries(symbol, {
    period1: "2019-01-01", type: "annual", module: "all",
  }),
  yf.quoteSummary(symbol, {
    modules: ["assetProfile", "price", "quoteType"],
  }),
]);
```

`Promise.all` runs both downloads **at the same time** rather than one after the other, which is about twice as fast. One brings the accounts, the other brings the company name, its sector and its size. We keep the last four years, oldest first.

Rule 1: missing stays missing

```
function num(value: unknown): number | null {
  return typeof value === "number" && Number.isFinite(value) ? value : null;
}
```

Every field can be a number **or** `null`. In programming, `null` means “there is no value here” — which is different from zero. Zero is a real amount. `null` means we were never told.

This is the most important rule in the whole file. Imagine a company does not report its total debts, and we treated that as 0. The debt-to-equity ratio would come out as 0.00, which reads as **this company has no debt at all** — the most reassuring message possible, produced from no information whatsoever.

Instead the app shows:

```
Debt to equity    Not reported  (total liabilities not reported)
```

That is a real example. NVIDIA genuinely does not report that figure through this data source.

Rule 2: flipping the sign on spending

```
function capexMagnitude(value: unknown): number | null {
  const n = num(value);
  return n === null ? null : Math.abs(n);
}
```

Yahoo reports money spent on buildings and equipment as a **negative** number, because it is cash leaving. But our formula is “cash from running the business minus money spent on equipment.”

If the spending arrived as −5,000 and we subtracted it, we would actually be **adding** 5,000 and reporting far too much spare cash. `Math.abs` makes it positive first. A small line of code preventing a large error.

Part 6 — The eleven ratios

First, three words from accounting:

- **Assets** — things the company owns: cash, stock, buildings.
- **Liabilities** — things it owes: bills, loans.
- **Equity** — assets minus liabilities. What would be left for the owners if everything were settled today.

The word **current** in front of any of these means **within the next twelve months**. Current assets become cash within a year. Current liabilities are bills due within a year.

Group 1: Liquidity — can it pay its bills?

Current ratio = current assets ÷ current liabilities

In plain words: for every £1 the company must pay in the next year, how many pounds does it have coming in during that year?

2.0 means it has twice what it needs. Below 1.0 means it owes more than it expects to receive — usually a warning, but completely normal for supermarkets, who take your money instantly and pay their suppliers two months later.

Quick ratio = (current assets − inventory) ÷ current liabilities

In plain words: the same question, but stricter. Inventory is unsold stock sitting in a warehouse. In an emergency you might not be able to sell it quickly, or you might have to sell it cheaply. So this version removes it and asks: can you pay your bills **without** relying on selling stock?

Group 2: Profitability — does it make money?

Gross margin = gross profit ÷ revenue

In plain words: out of every £100 of sales, how much is left after paying only for the thing you sold?

A software company keeps about £80, because making another copy costs almost nothing. A supermarket keeps about £25, because it has to buy everything it sells. Neither is better — they are different businesses.

Operating margin = operating income ÷ revenue

In plain words: out of every £100 of sales, how much is left after paying for the goods **and** running the business — wages, rent, advertising? This is a better test of whether the business actually works.

Net margin = net income ÷ revenue

In plain words: out of every £100 of sales, how much is real profit after absolutely everything, including tax and loan interest. The “bottom line”.

Return on equity = net income ÷ total equity

In plain words: for every £1 the owners have in the business, how much profit does it produce in a year? 20% means each £1 of owner money earns 20p a year.

This one has a trap, and the code handles it:

```
if (y.totalEquity !== null && y.totalEquity <= 0) {
  return { value: null, status: "not_meaningful", inputs };
}
```

If a company owes more than it owns, equity is a negative number. Dividing a profit by a negative number gives a negative answer — which reads as "losing money" when the company is actually profitable. The sum works; the meaning is nonsense. So the app refuses to show a number and says **Not meaningful (negative equity)** instead.

Group 3: Leverage — how much does it owe?

Debt to equity = total liabilities ÷ total equity

In plain words: for every £1 the owners put in, how much has the company borrowed? 0.5 means 50p borrowed per £1 of owner money — careful. 3.0 means £3 borrowed per £1 — much riskier, because loans have to be repaid whether or not you make a profit.

This is the only ratio where a smaller number is better, and that single fact caused two separate bugs. Both are in Part 8.

```
higherIsBetter: false,   // only debt_to_equity sets this
```

Equity ratio = total equity ÷ total assets

In plain words: out of everything the company owns, what percentage was paid for with the owners' own money rather than borrowed money? 60% means well over half the business is genuinely owned.

Group 4: Cash — does real money arrive?

Profit and cash are not the same thing. A company can record a sale as profit before the customer has actually paid. These ratios check that real money is coming in.

Free cash flow = operating cash flow − capital expenditure

In plain words: cash made by running the business, minus what had to be spent on equipment and buildings to keep it running. What is genuinely spare. If this is negative year after year, the company is burning money.

FCF margin = free cash flow ÷ revenue

In plain words: out of every £100 of sales, how many pounds end up as genuinely spare cash?

Operating cash ratio = operating cash flow ÷ current liabilities

In plain words: can the cash the business actually makes in a year cover the bills due that year? Stricter than the current ratio, because it uses real cash rather than things that **should** turn into cash.

How a ratio is written in code

Each ratio is one entry in a list. This is why adding a twelfth ratio means adding one entry and changing nothing else:

```
interface RatioDef {
  id: RatioId;              // "current_ratio"
  label: string;            // "Current ratio"
  category: Category;       // "liquidity"
  unit: Unit;               // "ratio" | "percent" | "currency"
  higherIsBetter?: boolean; // true unless stated; only debt_to_equity is false
  compute: (year: StatementYear) => {
    value: number | null;
    status: YearStatus;
    inputs: Record<string, number | null>;
  };
}
```

Division is always protected

```
function divide(numerator: number | null, denominator: number | null): number | null {
  if (numerator === null || denominator === null || denominator === 0) return null;
  return numerator / denominator;
}
```

Three dangers in one small function: a missing top number, a missing bottom number, and dividing by zero. In JavaScript, dividing by zero does not cause an error — it quietly gives `Infinity`, which would then be displayed. All three cases give `null` instead, which travels safely through the rest of the app.

Three possible outcomes, not two

Status	What it means	Shown on screen as
ok	The number was worked out successfully	The number
unavailable	Something was missing, or we divided by zero	“Not reported” plus the reason
not_meaningful	The sum worked but the answer misleads	“Not meaningful (negative equity)”

Most apps have only two states: we have it, or we do not. That third state is what stops the app showing a technically correct but misleading number.

And the reason is always given

```
const missing = entries.filter(([, v]) => v === null).map(([k]) => label(k));
if (missing.length > 0) return `${missing.join(", ")} not reported`;

// Every input WAS reported, so the only way the value is still null is a
// zero denominator. Saying "not reported" here would be false.
const zeros = entries.filter(([, v]) => v === 0).map(([k]) => label(k));
if (zeros.length > 0) return `${zeros.join(", ")} reported as zero`;
```

The app separates “the company did not tell us” from “the company told us it was zero.” Those are different facts, and it refuses to mix them up.

Part 7 — Deciding strong, adequate or weak

File: `lib/health.ts`

```
current_ratio: [
  { upTo: 1.0,      verdict: "weak" },
  { upTo: 1.5,      verdict: "adequate" },
  { upTo: 3.0,      verdict: "strong" },
  { upTo: Infinity, verdict: "adequate" },   // <-- notice this
],
```

Read the list from the top and stop at the first line the value fits. A current ratio of 2.0 is above 1.5 but below 3.0, so it lands on **strong**.

Why the scale comes back down

That last line is real financial thinking, not a mistake. A current ratio of 8.0 is **not** better than 2.5. It usually means the company is sitting on a mountain of cash doing nothing instead of investing it. So very high values drop back to **adequate**.

Debt to equity runs backwards

```
debt_to_equity: [
  { upTo: 0.5,      verdict: "strong" },     // little debt = good
  { upTo: 1.5,      verdict: "adequate" },
  { upTo: Infinity, verdict: "weak" },       // lots of debt = bad
],
```

The four summary boxes

```
const VERDICT_SCORE = { weak: 0, adequate: 1, strong: 2 };

const avg = scored.reduce((sum, v) => sum + VERDICT_SCORE[v], 0) / scored.length;
if (avg >= 1.5) return "strong";
if (avg >= 0.5) return "adequate";
return "weak";
```

Each ratio's verdict becomes a score, the scores are averaged, and the average becomes a word again. Liquidity has two ratios: if one is strong (2) and the other adequate (1), the average is 1.5, so the Liquidity box reads **strong**.

Nothing means nothing

```
if (scored.length === 0) return "unavailable";
```

If no ratio in a group could be worked out, the box says **unavailable**. It does not quietly settle on the middle option. Guessing “adequate” from no data would be inventing a verdict.

An honest note in the code

> These bands are judgment calls, not universal financial law — the same numbers get different readings in different sectors.

That comment is really in the file. The thresholds give a quick first opinion; the AI explanation underneath is where the industry context softens or overrules them.

Part 8 — Trends, and two real bugs

How a trend is worked out

```
const pctChange = ((last - first) / Math.abs(first)) * 100;
direction = pctChange > 5 ? "up" : pctChange < -5 ? "down" : "flat";
```

Compare the oldest year with the newest. If the change is less than 5% either way, call it **stable** — small wobbles are noise, not trends.

Bug 1: 'improving' did not mean improving

The first version said: number went up means improving. For ten of the eleven ratios that is correct. For debt to equity it is exactly backwards.

A company whose debt to equity climbed from 0.4 to 1.8 has borrowed heavily. The old code called that **Improving** — and, far worse, **put the word 'improving' into the prompt**, so the AI obediently wrote a paragraph about the company's strengthening finances.

> A one-word mistake in the data became a paragraph of confident nonsense. This is the clearest proof of why the maths must be right: **the AI repeats your mistakes fluently.**

```
const movedTheHealthyWay = direction === "up" ? higherIsBetter : !higherIsBetter;
return { trend: movedTheHealthyWay ? "improving" : "declining", direction };
```

Bug 2: the arrow pointed the wrong way

After fixing bug 1, the word `trend` meant **healthier**, not **bigger**. But the arrow on screen was still drawn from `trend`: improving showed an up arrow. Meanwhile the little chart beside it plots the **actual values**.

So on debt to equity the two disagreed. A company paying off its debts showed an **up** arrow labelled Improving, right next to a line sloping **down**.

The cause: the code worked out which way the number moved, then threw that away and kept only the verdict. The part that draws the arrow had to guess, and for one ratio the guess was wrong.

The fix: keep both, and let each one drive the thing it actually describes.

What you see	Driven by	Meaning
The arrow	direction	Which way the number moved
The word and colour	trend	Whether that was good news
The little chart	raw values	The real four-year shape

Now a company reducing its debt shows a **down arrow, the word Improving, in green**, beside a downward line. Everything agrees. Checked on Apple, whose debt to equity has fallen across all four reported years.

Part 9 — The handwritten notes

The AI is not allowed to rely on what it remembers. It has to argue from material we control: **42 notes** in `content/kb/`, written by hand.

What a note looks like

```
---
id: current-ratio-sector-exceptions
ratio: current_ratio
tags: [liquidity, sector, retail, software]
---

Grocery and restaurant chains collect cash from customers immediately but
pay suppliers on 30-60 day terms, so they can run current ratios below 1.0
indefinitely without distress. Walmart and Costco have both operated for
years with ratios near 0.8-0.9.
```

The section between the dashes is called **frontmatter** — labels about the note. The rest is the knowledge itself. The notes cover what each ratio means, industry exceptions, common traps, and honest warnings about what ratios cannot tell you.

The problem: which three notes?

Sending all 42 notes every time would be slow and expensive. We need the three **most relevant** notes for this ratio, this company, this industry.

Turning text into numbers

An **embedding** turns a piece of text into a long list of numbers — 1,024 of them here — that represents its meaning. Texts that mean similar things get similar lists.

Think of it as giving every sentence a position on a huge map of meaning. Sentences about supermarket cash land near each other. Sentences about software profits land somewhere else. To find relevant notes, you see which ones sit closest to your question on that map.

The closeness score is called **cosine similarity**. It runs from −1 (opposite) to 1 (pointing the same way).

Done once, in advance

```
npm run build-kb-index
```

This reads all 42 notes, sends them to Jina to be turned into numbers, and writes `content/kb-index.json`. **That file is saved into the project.** So the live website never has to process the notes again — it only has to process the short search question, which is fast and almost free.

The search question uses everything

```
`${ratioLabel} ${formattedValue} ${trend} trend ${sector} sector ${category}`
// -> "current ratio 0.89 declining trend consumer defensive sector liquidity"
```

Notice it is not just “current ratio”. Putting the value, the trend and the industry into the question is what lets the **supermarket exception** note beat the plain definition when the company is Walmart.

The pinning fix

```
export function pinOwnNote(ctx, ranked, all, k) {
  if (ranked.some((n) => n.ratio === ctx.ratioId)) return ranked;
  const own = all.find((n) => n.ratio === ctx.ratioId);
  if (!own) return ranked;
  return [own, ...ranked].slice(0, k);
}
```

Searching by similarity alone had a quiet failure. On Apple and Home Depot, the gross-margin note **lost all three places** to broader industry notes. Without its own note, the AI fell back on figures it half-remembered — and the checker correctly deleted the explanation, because those figures were never given to it.

Now the ratio's own note is guaranteed a place, and the search fills the other two.

Two safety nets

Net 1: send them all together

A report has up to eleven ratios. Sending eleven separate requests at the same moment broke Jina's free limit on how many you may send at once. Now all the search questions go in **one** request, with a counter called a **semaphore** as a backup that lets only two requests run at a time.

Net 2: fall back to keywords

If the whole number-conversion service fails, the app does not give up. It matches on the labels instead, where an exact ratio match scores +3 — more than any combination of labels can reach:

```
const ratioBonus = note.ratio === ctx.ratioId ? 3 : 0;
```

> Bad search settings should cost you search quality, nothing more.

That is why this whole section sits inside a `try` block. Before, a settings mistake escaped and wiped out every explanation on the page.

Part 10 — Talking to the AI

File: `lib/llm.ts` — **the only file in the whole project allowed to talk to an AI service.** Everything else goes through its one function.

Why that rule helps

Groq, Cerebras, Mistral and several others all accept requests in the same format. Because only one file touches that, switching to a different AI provider is three settings — never a code change.

```
LLM_BASE_URL=https://api.groq.com/openai/v1
LLM_API_KEY=your-key-here
LLM_MODEL=openai/gpt-oss-120b
```

The backup provider, and the bug that disabled it

```
function isRetryable(err: unknown): boolean {
  if (err instanceof OpenAI.APIError) {
    if (err.status === 404) return true;                                    // added
    if (err.status === 400 && /model/i.test(err.code ?? "")) return true;  // added
    return err.status === 429 || (err.status !== undefined && err.status >= 500);
  }
  return true;   // no reply at all = network problem = worth retrying
}
```

The original version only retried on **429** (too many requests) and **5xx** (the provider's own fault). Then Groq retired the AI model we were using. A retired model replies **404**, which matched nothing, so the backup never ran.

> The feature broke on exactly the day a backup provider exists for. Three months passed before anyone noticed, because a missing explanation looks identical to a missing password.

The prompt

One request per **group** of ratios, not per ratio — four requests per report instead of eleven. The AI is asked to reply in a strict format so each paragraph can be matched back to the right row:

```
Respond with ONLY a JSON object mapping each ratio id to its explanation
string - no markdown code fences, no commentary outside the JSON. Keys must
be exactly: "current_ratio", "quick_ratio".
```

Part 11 — The number guard

The most distinctive part of the project, and the best thing to explain in an interview.

The rule

> Find every number in what the AI wrote. If a number was never given to the AI, the AI invented it — so throw that explanation away.

```
const NUMBER_PATTERN = /-?\d+(?:,\d{3})*(?:\.\d+)?%?/g;
```

That pattern finds negatives, commas, decimals and percentages: `1.84`, `-12.5`, `1,250`, `60%`.

The hard part: what counts as 'given'?

The first version allowed only the exact ratio values. It deleted nearly every good explanation. Four fixes were needed, each one discovered by watching real output.

Fix 1 — ordinary words that contain numbers

```
const RHETORICAL_CONSTANTS = [0, 1, 100];
```

The checker threw away a perfectly correct explanation because of the **1** in "$7.20 of debt for every $1 of equity." That 1 is part of the sentence, not a financial claim. Nothing can be faked using 0, 1 or 100.

Fix 2 — all four years, not just the newest

The prompt gives a trend and asks what it suggests, which naturally produces "from 2022 to 2025." With only 2025 allowed, that correct sentence was binned. A year is a date, not a financial claim.

Fix 3 — the number as the AI actually saw it

```
for (const n of extractNumbers(formatRatioValue(r))) allowed.add(n);
```

Free cash flow is stored as `108800000000`, but the prompt **shows** the AI `$108.8B`. So the AI writes "108.8", which matched neither the stored value nor any year. The result: **every single money explanation was deleted, every single time.**

Fix 4 — the same number in different units

```
if (r.unit === "ratio") allowed.add(latest.value * 100);
else if (r.unit === "percent") allowed.add(latest.value / 100);
```

AI models constantly restate a ratio as a percentage: "an operating cash ratio of 0.85 … covers 85% of current liabilities." 0.85 and 85% are **one number in two units**, not two claims.

And rounding — the biggest cause of all

```
function matchesAllowed(written: number, allowed: number): boolean {
  if (Math.abs(allowed - written) < 0.05) return true;
  const decimals = (String(written).split(".")[1] ?? "").length;
  return Number(allowed.toFixed(decimals)) === written;
}
```

AI models round numbers to make sentences readable: 12.7% becomes “roughly 13%”; 8.6% becomes “about 9 pence in the pound.” Treating that as invention was the single biggest reason ratios appeared with no text at all.

The clever part: the rounding is applied to the **allowed** number, to however many decimal places the AI actually used. So 12.7 rounds to 13 and matches. But 2025 never rounds to 2015, and 1.84 never rounds to 4.2.

> A wrong number stays wrong at every level of rounding. The checker became relaxed about **how a number is written** without becoming relaxed about **which numbers are true**.

What happens when it catches something

```
const offending = findDisallowedNumber(text, allowed);
if (offending !== null) {
  console.warn(`dropped explanation for ${r.id} - number guard rejected ${offending}`);
  continue;      // this one ratio loses its text; the others keep theirs
}
```

It removes **one ratio's** paragraph, keeps the rest, and writes the offending figure into the log so you can see what happened. This is why Home Depot sometimes shows a gross margin with no text: the AI reached for a remembered industry figure. That is **the checker working, not failing**.

Part 12 — Saving answers

File: `lib/cachedData.ts`. Two saved stores, deliberately built differently.

Store 1: the company data

Saved under the company's ticker, thrown away after 24 hours. Company accounts change four times a year at most, and the data source is free and unofficial, so downloading it on every page view would be both wasteful and risky.

Store 2: the AI explanations

```
const payload = JSON.stringify({
  company, sector, category,
  ratios: ratios.map(r => ({ id: r.id, value: r.latestValue,
                             status: r.latestStatus, trend: r.trend }))
              .sort((a, b) => a.id.localeCompare(b.id)),
});
return createHash("sha256").update(payload).digest("hex");
```

This one is saved under a **fingerprint of the numbers themselves**, not under the company name and date. The technical term is **content addressing**. It is a neat trick:

- Same numbers means the same fingerprint, so we reuse the saved answer.
- Any number changes and the fingerprint changes, so we write a fresh one.
- So this store needs **no expiry date at all**. It cannot go out of date, because out-of-date numbers would produce a different fingerprint.

Failures are never saved

```
try {
  return await cached();
} catch (err) {
  console.error(`FinPulse: NO EXPLANATIONS for category "${input.category}" - ` +
    `ratios will render bare. Usually a bad or missing LLM_* setting, or a ` +
    `model the provider has retired. Cause:`, err);
  return {};
}
```

If the AI request **fails** — no password set, dead model, network down — that failure is never saved. Otherwise one request made before you added your keys would poison the store permanently.

A checker rejection is different: that is a finished, repeatable result, so it is saved normally.

> The page gives no hint: it shows the ratios with no text underneath, which looks exactly the same as 'no password set'. That ambiguity is how a retired model went unnoticed for three months.

That is why the error message is now so loud and so specific. The lesson: **failing quietly is worse than crashing**, because nobody investigates something that looks normal.

Part 13 — The screen

The page arrives in two pieces

The numbers are pure arithmetic, ready in milliseconds. The AI text takes several seconds. Rather than make you stare at a blank screen, the app sends the numbers immediately and adds the text when it arrives.

```
<Suspense fallback={<RatioList ratios={ratios} explanations={pending} />}>
  <CategoryExplanations category={category} ratios={ratios} ... />
</Suspense>
```

`fallback` is what shows while the slow part is still coming. The clever detail: the fallback is **the same component**, just with “Generating…” where the text will go. So when the real text arrives, nothing on the page moves. No jumping.

Dark mode

```
const THEME_SCRIPT = `try{var m=localStorage.getItem("theme");
if(m==="light"||m==="dark"){document.documentElement.setAttribute("data-theme",m)}}
catch(e){}`;
```

Three deliberate details:

1. It runs **before the page is drawn**. React runs after the first draw, so doing this inside a component would flash the wrong colours for a moment — a white flash for anyone who chose dark mode.
2. It is wrapped in `try/catch`, because the browser's storage **throws an error** in private browsing. Without the catch, the whole page would break for those users.
3. It only accepts the exact words “light” and “dark”, so nothing unexpected can be pushed into the page.

The little chart

```
const y = HEIGHT - PAD - ((p.value - min) / range) * (HEIGHT - PAD * 2);
```

Drawn by hand, no charting library. The one thing to notice: on a screen, **y gets bigger as you go down**. So this line flips the value — a bigger number produces a smaller y, which draws higher up. With fewer than two years of data it draws a dashed line instead of a misleading single dot.

Part 14 — Testing

What is a test harness?

A **test harness** is the machinery that finds your tests, runs them, gives them the tools to check answers, and reports what passed and failed. You write the tests; the harness runs them.

In this project the harness is Vitest. It gives us `describe`, `it` and `expect`, finds every file ending in `.test.ts`, runs them all, and prints a summary.

```
import { describe, expect, it } from "vitest";

describe("a group of related tests", () => {
  it("says what should be true", () => {
    expect(somethingWeCalculated).toBe(theAnswerWeExpect);
  });
});
```

`describe` groups tests. `it` is one test, named in plain English. `expect` is the check. If the check fails, that test is reported as failed and the others still run.

The settings

```
export default defineConfig({
  test: { environment: "node" },
  resolve: { alias: { "@": path.resolve(__dirname, ".") } },
});
```

`environment: "node"` means tests run as plain programs with no pretend browser. That is faster, and correct here because everything we test is pure logic. The alias lets a test write `@/lib/ratios` exactly as the app does.

The 77 tests

File	Tests	What it proves
`ratios.test.ts`	35	Every formula, every missing-data case, trends, direction
`explain.test.ts`	24	The number guard: what it allows and what it rejects
`health.test.ts`	7	The threshold bands and the averaging
`format.test.ts`	6	The ‘why is this missing’ messages
`retrieve.test.ts`	5	Building the search question and pinning notes

What the ratio tests check

- Each formula gives the right answer from known inputs.
- A missing input gives `null` and “unavailable”, never 0.
- Dividing by zero gives `null`, not `Infinity`.

- Negative equity gives “not meaningful” for the two ratios affected.
- Equity ratio still works with negative equity, because it divides by assets.
- Spending is made positive before the free-cash-flow subtraction.
- Trends need two usable years; years with gaps are skipped.
- Movement under 5% reads as stable.
- **Rising debt to equity is ‘declining’, not ‘improving’.**
- **The arrow direction and the word deliberately disagree on debt to equity.**

What the checker tests check

These are the most interesting tests in the project, because nearly every one exists because of a real failure that was actually seen:

```
it("passes an explanation that only uses given numbers", () => {
  const text = "In 2024 the current ratio was 1.84, within the comfortable 1.5 to 3.0 range.";
  expect(passesNumberGuard(text, allowed)).toBe(true);
});

it("fails an explanation that invents a number nowhere in the context", () => {
  const text = "This is far below the industry average of 4.2, which is concerning.";
  expect(passesNumberGuard(text, allowed)).toBe(false);
});

it("keeps 12.7% restated as roughly 13%", () => {
  expect(passesNumberGuard("An operating margin of roughly 13% is typical.", allowed)).toBe(true)
;
});

it("does not let rounding turn a wrong number into a right one", () => {
  expect(passesNumberGuard("An operating margin of 48% is strong.", allowed)).toBe(false);
});
```

Notice the pattern. Every time the checker was made more relaxed, a test was added proving the relaxation did not break it. That pairing is the whole discipline, and it is a very good thing to be able to describe.

Every command you need

Command	What it does
`npm test`	Runs all 77 tests once and stops
`npx vitest`	Watch mode — re-runs tests as you edit files
`npx vitest --ui`	Opens a browser view showing each test
`npx vitest run ratios`	Runs only the files matching ‘ratios’
`npx vitest run --reporter=verbose`	Prints every test name, not just the totals
`npx tsc --noEmit`	Checks all the types without building anything
`npx eslint`	Checks code style and common mistakes

Command	What it does
`npm run build`	Full production build, which also checks types

What a good run looks like

```
 RUN  v4.1.10 F:/projects/FinPulse-financial-health-explainer-main

 Test Files  5 passed (5)
      Tests  77 passed (77)
   Duration  5.48s
```

A note for this computer

If your `C:` drive is full, `npx` fails with an error about no space. Until you clear it, run the local programs directly and send temporary files to `F:`:

```
$env:TMP="F:\tmp"; $env:TEMP="F:\tmp"
node node_modules/vitest/vitest.mjs run
node node_modules/typescript/bin/tsc --noEmit
node node_modules/next/dist/bin/next build
```

`npm cache clean --force` usually frees several gigabytes.

Part 15 — Running it yourself

Step 1 — install

```
cd F:\projects\FinPulse-financial-health-explainer-main
npm install
```

Step 2 — get two free keys

Setting	Where to get it	What it is for
`LLM_API_KEY`	console.groq.com	Writing the explanations
`EMBED_API_KEY`	jina.ai/embeddings	Searching the handwritten notes

Both are free and neither asks for a card.

Step 3 — create .env.local

Copy `.env.example` to `.env.local` and put the keys in the copy. `.env.local` **is kept out of the project history;** `.env.example` **is not** — so never put a real key in the example file.

```
LLM_BASE_URL=https://api.groq.com/openai/v1
LLM_API_KEY=gsk_your_key_here
LLM_MODEL=openai/gpt-oss-120b

EMBED_PROVIDER=jina
EMBED_API_KEY=jina_your_key_here
EMBED_MODEL=jina-embeddings-v3
```

Step 4 — run it

```
npm run dev            # then open http://localhost:3000
```

Step 5 — try these

```
http://localhost:3000/AAPL     Apple
http://localhost:3000/NVDA     NVIDIA  - debt to equity shows "Not reported"
http://localhost:3000/WMT      Walmart - low current ratio, sector exception
http://localhost:3000/HD       Home Depot
```

Other commands

Command	What it does
`npm run build-kb-index`	Rebuild the note index after editing any note
`npm run build`	Production build

Command	What it does
`npm start`	Serve the production build

If the explanations do not appear

1. Look at the terminal. The app prints a loud `NO EXPLANATIONS` message with the reason.
2. Check both keys are in `.env.local`, not `.env.example`.
3. Check `LLM_MODEL` is a model the provider still runs. Retired models reply 404.
4. If you changed `EMBED_MODEL`, re-run `npm run build-kb-index`, or the saved index no longer matches and the search quietly gets worse.

Part 16 — Putting it online

The site is hosted on **Vercel**, connected to the GitHub repository. Every time you push code to the `main` branch, Vercel rebuilds and publishes it automatically.

```
git add <the files you changed>
git commit -m "your message"
git push origin main
# Vercel notices and redeploys on its own
```

The two keys are set in the Vercel dashboard under Environment Variables, not in the code. That is why the live site works even though `.env.local` was never uploaded.

One known limit

The four requests are sent at the same moment, roughly 8,800 units of text against Groq's free limit of 8,000 a minute. On a company nobody has viewed before, one group can be refused and will appear without text. Visiting popular companies once each saves their answers permanently and avoids the problem.

Part 17 — Is this ‘harness engineering’?

Short answer: **the project has a test harness, but it does not use Harness the product.** Three different things share that word, so here is each one.

1. A test harness — yes, we have one

The machinery that finds, runs and reports on tests. Ours is **Vitest**, set up in `vitest.config.ts`. If someone asks “what test harness do you use?”, the answer is Vitest, with 77 tests in 5 files.

2. Harness.io — no, not used

Harness is a paid service for automatically testing and publishing code. It competes with Jenkins and GitHub Actions. This project does not use it.

What we use instead is **Vercel's GitHub connection**: push to `main`, and Vercel builds and publishes. That is the same idea, much simpler.

3. ‘Agent harness’ — yes, in a real sense

In AI work, a **harness** sometimes means all the code wrapped around a language model: the part that builds the prompt, supplies reference material, checks the output and handles failures. The model on its own does nothing useful; the harness turns it into a product.

By that meaning, this project is mostly harness — and it is the most interesting part:

Harness job	Where it lives
Build the prompt from computed data	lib/explain.ts
Supply real reference material	lib/retrieve.ts, the 42 notes
Limit what the model may do	The rules section of the prompt
Check the output	The number guard
Handle the provider failing	lib/llm.ts, the backup provider
Avoid repeating work	lib/cachedData.ts
Fail safely	Keyword fallback, bare ratios, loud logs

How to answer it out loud

> “Our test harness is Vitest — 77 tests covering the calculation layer and the output checker. We don't use Harness.io; publishing is handled by Vercel's GitHub integration. But if you mean the harness **around the model** — building the prompt, supplying retrieved notes, checking the output and handling provider failure — then yes, that's most of what the project actually is.”

Part 18 — Interview questions

Grouped by what the interviewer is really testing. Written the way you would actually say them.

A. The overview

> **Q.** Tell me about this project.

“FinPulse explains a company's financial health in plain English. You type a ticker, it downloads four years of real accounts from Yahoo Finance, works out eleven standard ratios, judges each as strong, adequate or weak, and then uses an AI model to write a short explanation under each one.”

“The core principle is that our code does all the arithmetic and the model only writes language. The model never calculates — it receives finished numbers. And there's a checking layer called the number guard that reads what the model wrote, pulls out every number, and throws away any explanation containing a figure the model wasn't given.”

“It's Next.js and TypeScript, hosted on Vercel, with 77 tests in Vitest.”

> **Q.** Why not just give the AI the accounts and ask it to analyse them?

“Because models invent numbers, and in a finance app a confident wrong number is worse than no answer. If it says a margin is 40% when it's 4%, you'd believe it.”

“So I split the work. The arithmetic is ordinary code — same input, same answer, fully testable. The model does the one thing it's genuinely good at, turning a correct number into a readable sentence. Then the checker verifies it didn't drift.”

B. Architecture

> **Q.** Walk me through what happens when someone types a ticker.

“The form redirects to a page for that ticker. On the server we download four years of accounts and the company profile in parallel, compute eleven ratios for each year, work out trends and verdicts, and send that to the browser straight away — so the user sees numbers almost immediately.”

“Then, behind a Suspense boundary, we take those computed results and use them to search our knowledge base for the three most relevant notes per ratio. The numbers and the notes go into one prompt, the model writes the explanations, the number guard checks them, and the text streams in underneath.”

“The important ordering detail is that the computation happens before the search, because the computed value and trend are part of the search query.”

> **Q.** Why is lib/ kept separate from the website code?

“Everything in `lib/` is plain TypeScript with no framework dependency. Only `lib/cachedData.ts` imports from Next, because the caching helper needs Next's request context.”

“The practical benefits are that tests can import the logic directly with no framework setup, and the same functions can be called from a plain script. Keeping framework code at the edges is a habit that keeps paying off.”

> **Q.** How do you handle missing data?

“Every field from the data source is typed `number | null`, and null is never turned into zero. That's the single most important rule in the data layer.”

“If a company doesn't report total liabilities and you treated that as 0, debt to equity would read 0.00 — which says 'no debt at all', the most reassuring message possible, produced from nothing.”

“There's also a third state beyond available and missing: 'not meaningful'. That's for when the arithmetic works but misleads — return on equity with negative equity, for instance. A profitable company with negative equity gives a negative return on equity, which reads as a loss. So we refuse to show a number and say why instead.”

> **Q.** Explain your caching strategy.

“Two stores with different designs. Company data is saved under the ticker with a 24-hour expiry, because accounts change quarterly at most and the source is unofficial.”

“The explanation store is more interesting. It's saved under a hash of the actual ratio values that will go into the prompt — content addressing. Identical numbers produce an identical prompt, so they can safely share an answer, and any change to a number produces a new key automatically. So it needs no expiry at all; it can't go stale.”

“The key detail is that failures are never saved. Otherwise one request made before the keys were configured would poison the store forever. A checker rejection is different, because that's a finished repeatable result.”

C. The AI layer

> **Q.** What is RAG and how did you implement it?

“RAG is retrieval-augmented generation. Instead of letting the model rely on what it memorised during training, you find relevant reference material and put it in the prompt, so it argues from your sources.”

“Mine is 42 handwritten notes covering ratio definitions, industry exceptions and common misreadings. At build time each note is turned into a 1,024-number vector and saved into a JSON file that's committed to the repo, so the live app never re-processes the notes — only the short search query.”

“At request time I build a query from the full situation: ratio name, the actual value, the trend, the sector, the category. That gets embedded and compared against all 42 notes by cosine similarity, and I take the top three. Putting the sector in the query is what lets a supermarket-exception note beat a plain definition when the company is Walmart.”

> **Q.** Did plain similarity search work?

“Not completely, and the failure was quiet. On Apple and Home Depot the gross-margin note lost all three places to broader industry notes. Without its own note the model fell back on a half-remembered benchmark, and the number guard correctly deleted the whole explanation because that figure was never supplied.”

“So I pin the ratio's own note into the result and let similarity fill the other two slots. The point of a knowledge base is that the model argues from it, and similarity alone didn't guarantee that.”

> **Q.** Explain the number guard.

“It pulls every number out of the generated text and checks each one against the set of numbers the model was given. If any number isn't accounted for, the explanation is dropped.”

“The hard part was defining 'given'. The naive version allowed only the exact ratio values and destroyed almost everything legitimate. It needed four fixes: allow 0, 1 and 100 as parts of ordinary sentences; allow every fiscal year in the report; allow the formatted version of a number, because free cash flow is stored as 108800000000 but shown to the model as $108.8B; and allow unit restatement, because 0.85 and 85% are one number in two units.”

“And rounding, which was the biggest cause. Models write 'roughly 13%' for 12.7%. The trick is to round the allowed value to whatever precision the model used. So 12.7 matches 13, but 2025 never matches 2015 and 1.84 never matches 4.2. It got relaxed about form without getting relaxed about fact.”

> **Q.** What happens when the guard rejects something? Does the page break?

“No. It drops that one ratio's paragraph, keeps the rest of the section, and logs the offending figure so it's diagnosable. The user sees the number and the verdict with no text underneath — worse, but never wrong.”

“That's the philosophy: silence is acceptable, invention is not.”

> **Q.** How do you handle the AI provider failing?

“There's a main provider and a backup, both using the same request format, so switching is configuration rather than code. The retry logic covers rate limits, server errors, network failures and — this is the one I got wrong at first — 404s.”

“I'd only handled 429 and 5xx. Then Groq retired the model we were using. A retired model replies 404, which matched nothing, so the backup never fired. The feature died on exactly the day a backup exists for, and it took three months to notice because a missing explanation looks identical to a missing key. That's why the error message is now extremely loud and names the likely cause.”

D. Debugging

> **Q.** Tell me about a bug you found and fixed.

“My favourite is the trend direction bug, because it's really two bugs teaching different lessons.”

“The first version equated 'the number went up' with 'improving'. That's right for ten of the eleven ratios, but debt to equity is inverted — lower is healthier. So a company whose debt ratio climbed from 0.4 to 1.8 was

labelled Improving. Worse, that word went into the prompt, so the model wrote a paragraph about strengthening finances. A one-word data error became a paragraph of confident nonsense. The lesson is that the model amplifies whatever you hand it."

"The second bug came from fixing the first. Once 'trend' meant 'healthier', the arrow on screen was still drawn from it, so improving meant an up arrow. But the little chart beside it plots raw values. On debt to equity they contradicted each other: an up arrow next to a line sloping down."

"The root cause was that the trend function calculated the raw direction and then discarded it. I now return both, and each drives the thing it actually describes — the arrow follows the number, the word and colour follow the health verdict."

> **Q.** How did you track down the second bug?

"A user reported the arrow looking wrong. I traced what each element was bound to — the arrow to the verdict, the chart to raw values — then checked which ratios could possibly disagree. Only one has the inverted flag, so it had to be debt to equity."

"Then I verified it rather than trusting the reasoning: built the app, served it, and inspected the rendered output for that row."

> **Q.** Why not just flip the chart for inverted ratios?

"Because the chart's job is to show the real data. Flipping it would make the chart lie so the arrow looked right — trading a confusing display for a dishonest one."

"The right fix was to stop mixing two different meanings and let each element show what it actually represents."

E. Testing

> **Q.** How do you test this?

"77 tests in Vitest across five files. The strategy is to test heavily where correctness is objective, and not to chase coverage where it isn't."

"So the calculation layer is tested thoroughly — every formula, every missing-input path, zero denominators, negative equity, trend thresholds. The number guard has 24 tests of its own."

"What I don't test is the model's actual wording, because it isn't repeatable. I test the **checker** instead. That's the real benefit of having a checker: it turns an untestable thing into a testable one."

> **Q.** Give an example of a test that exists because of a real bug.

"Almost all the number-guard ones. For example, 'keeps 12.7% restated as roughly 13%' exists because rounding was silently deleting most explanations. And immediately after it, 'does not let rounding turn a wrong number into a right one', which checks that 48% is still rejected."

"That pairing is the discipline. Every time I relaxed the checker, I added a test proving the relaxation didn't break it."

> **Q.** What test harness do you use?

“Vitest. It finds the `.test.ts` files, gives us describe, it and expect, and runs everything as plain Node with no browser simulation, because everything under test is pure logic.”

“If you mean Harness.io specifically — no. Publishing is handled by Vercel's GitHub integration, which rebuilds on every push to main.”

F. Trade-offs

> **Q.** What would you do differently?

“Three things. The four requests fire at the same moment and can exceed the free rate limit on a cold page, so one section renders bare. Sequencing them would fix it at the cost of a slower first load.”

“Second, the guard forbids all arithmetic, so it rejects 'the equity ratio is 12.2%, so the other 87.8% is debt'. That's correct and useful, but allowing it means permitting one specific subtraction, and I'd think hard before loosening a guard whose value comes from being strict.”

“Third, I'd add a proper pipeline so the tests gate deployment. Right now they run locally and Vercel builds on push.”

> **Q.** What's the weakest part?

“The data source. The library wraps an unofficial endpoint, so which fields exist is inconsistent. NVIDIA doesn't report total liabilities through it, so debt to equity is simply unavailable for one of the best-known companies on the list.”

“The app handles it honestly by naming exactly which input was missing, but no amount of good error handling fixes not having the data. A paid data source would be the real answer.”

> **Q.** How would you scale this to 10,000 users a day?

“The explanation store already does most of the work, because it's keyed by content — once a company has been viewed, everyone after that gets the saved text for free. The cost is only on the first view of each company.”

“Beyond that: move the store to something shared like Redis rather than per-server, pre-load the popular few hundred tickers on a schedule, sequence the requests to stay under rate limits, and move to paid tiers for both the model and the data.”

> **Q.** Why Groq rather than OpenAI?

“Cost and speed. Groq's free tier is generous enough for this and the responses are very fast, which matters because four requests fire per report.”

“But the design doesn't care. `lib/llm.ts` is the only file that talks to an AI service, and every provider I'd consider accepts the same format, so switching is three settings. That was deliberate — I didn't want the

provider baked into the code.”

G. Quick answers

Question	Answer
How many ratios?	11, in 4 groups
How many handwritten notes?	42
How big is an embedding?	1,024 numbers
How many notes per request?	3 — the ratio's own note is pinned
How many AI calls per report?	4, one per group
How many tests?	77 in 5 files
How many years of data?	4, oldest first
When is a trend 'stable'?	Movement of 5% or less
Expiry on saved explanations?	None — keyed by content
Expiry on saved company data?	24 hours
Which ratio is inverted?	Debt to equity only
Guard tolerance?	0.05, plus precision-aware rounding

Part 19 — Glossary

Word	What it means
API	A way for one program to ask another program for data
Cache	Saving an answer so you do not have to work it out again
CI/CD	Automatically testing and publishing code when it changes
Content addressing	Naming a saved answer after the data inside it
Cosine similarity	A score from −1 to 1 for how alike two lists of numbers are
Embedding	A list of numbers representing what a piece of text means
Frontmatter	Labels at the top of a text file, between dashes
Hallucination	When an AI confidently states something untrue
LLM	Large Language Model — the AI that writes the text
null	‘No value here’ — deliberately different from 0
Prompt	The instructions and data we send to the AI
RAG	Finding relevant notes and putting them in the prompt
Rate limit	A cap on how much you may request per minute
Semaphore	A counter that limits how many things run at once
Server Component	A piece of the page built on the server, not the browser
Sparkline	A tiny chart with no labels, showing shape only
Streaming	Sending part of a page before the rest is ready
Suspense	React's way of showing a placeholder while data loads
Test harness	The machinery that finds, runs and reports on tests
Ticker	A company's short stock-market code, such as AAPL
Token	About three quarters of a word, as an AI counts text
TTL	Time to live — how long a saved answer stays valid
Vector	An ordered list of numbers

> Yahoo gives us the raw numbers; missing stays missing and never becomes zero. Our code works out eleven ratios across four years. Fixed thresholds turn those into verdicts, honestly labelled as judgement calls. The computed result is then used to search 42 handwritten notes, and the best three are merged with the numbers into one prompt. The AI writes from that and nothing else. A checker reads every digit back and deletes anything that was not handed over.
>
> **It is a calculator with a writer attached, and a fact-checker standing between them.**